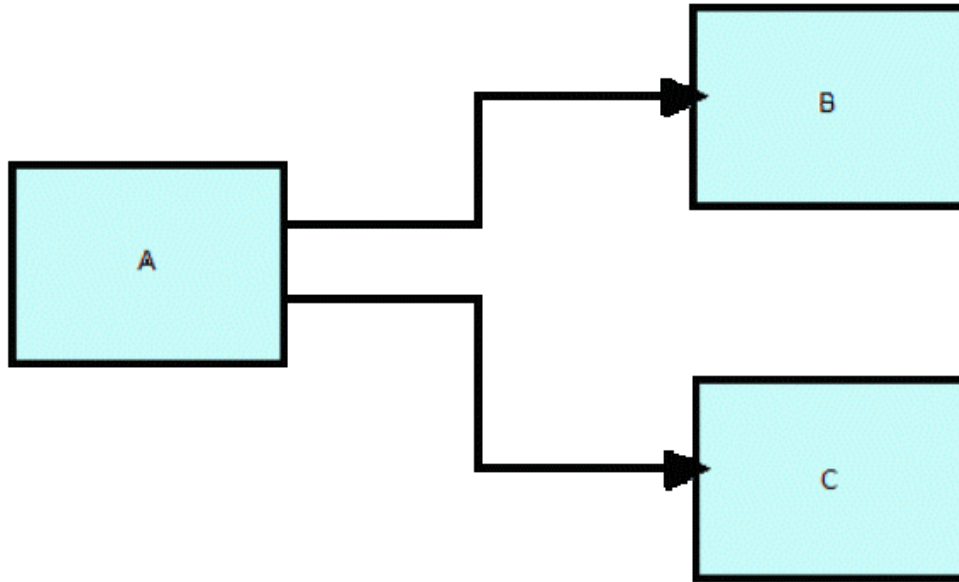


Flow-based Programming

Πάντα ρεῖ (*Panta rhei*)- Everything flows.



"The real world is asynchronous: don't try to force everything into a synchronous framework derived from the architecture of early computers."

In computer programming, Flow-Based Programming (FBP) is a programming paradigm, discovered/invented by [J. Paul Rodker Morrison](#) in the late '60s, that uses a "data processing factory" metaphor for designing and building applications. FBP defines applications as networks of "black box" processes, which communicate via data chunks (called Information Packets) travelling across predefined connections (think "conveyor belts"), where the connections are specified externally to the processes. These black box processes can be reconnected endlessly to form different applications without having to be changed internally. FBP is thus naturally component-oriented.

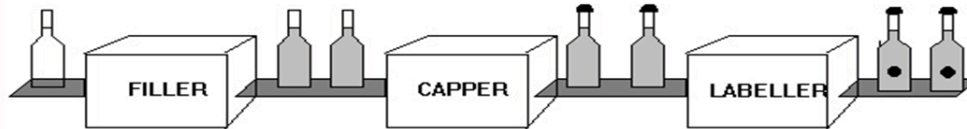
FBP is a special case of [dataflow](#) programming characterized by asynchronous, concurrent processes "under the covers", Information Packets with defined lifetimes, named ports, "bounded buffer" connections, and definition of connections external to the components - it has been found to support improved development time and maintainability, reusability, rapid prototyping, simulation, improved performance, and good communication among developers, maintenance staff, users, systems people, and management - not to mention that FBP naturally takes advantage of multiple cores... without the programmer having to struggle with the intricacies of multitasking!

FBP exhibits "data coupling", described in the Wikipedia article on the IT concept called "[coupling](#)" as the loosest type of coupling between components. Loose coupling provides significantly improved maintainability in systems.

FBP promotes a high-level, functional style of specifications that simplifies reasoning about system behavior.



Soft Drink Bottling Factory



- Independent well-defined functions
- Clean interfaces
- Simple to reconfigure
- Minimizes side-effects
- Designer can sit at one “station”, or can follow an item through system

7

FBP involves a significant "paradigm shift" from conventional programming, as it uses an "assembly line" image of data processing, which really has to be used to experience its advantages. Not surprisingly given the hold that the so-called "[von Neumann](#)" paradigm has over programming technology, over the last 40+ years a number of companies have come out with FBP-like or "FBP-inspired" systems (to borrow a term coined by Joe Witt of Cloudera), which exhibit some of the attributes of FBP, but do not involve this paradigm shift, including some of the biggest and most successful companies in the industry, such as IBM, Microsoft, Facebook, Google, and the New York Times (see [FBP External References](#)). Unfortunately, without the FBP paradigm shift, these systems cannot fully achieve the advantages listed above, although most of them combine a visual notation with some degree of componentry and "[configurable modularity](#)", to use the term popularized by the late Nate Edwards. We have therefore been forced to use the term "classical FBP" to refer to the FBP architecture described in this web site. It is hard for someone reading documentation to tell whether a given product describing itself as "an FBP system" is "classical" or not, but, roughly, multiple input ports, "back pressure", and asynchronism, constitute a sort of litmus test to check whether systems should be classed as "classical" FBP systems. While the FBP paradigm shift may seem unfamiliar, given the prevalence of the older programming approaches, in fact it embodies a simple mental model (not "simple" as in easy but "simple" as in elegant, to quote Joe Witt), and has proven over several decades to result in applications that are more understandable and maintainable than those built using the conventional (von Neumann) paradigm.

If you *have* come across Flow-Based Programming by way of NoFlo, Flowhub, Node-RED, or similar packages, it is recommended that you read [FBP vs. FBP-inspired Systems](#), which attempts to highlight the differences and similarities between FBP proper, or "classical" FBP, (the subject of this web site) on the one hand, and NoFlo and other similar "FBP-inspired" systems on the other.

FBP is described in J. Paul Morrison's book, [Flow-Based Programming: A New Approach to Application Development](#)", now in its 2nd edition, which can be obtained in paperback form from [Amazon](#), or in 2 e-book formats: [Kindle](#) and [epub](#).

Further down on this web page you will find a bunch of useful links, including some videos demonstrating the use of the FBP diagramming tool, [DrawFBP](#). During FBP's early days, networks were coded by hand, and initially we used pencil and paper, followed by more sophisticated graphical tools, whether or not they played well with FBP software. However, with the advent of good graphical support, we now have a powerful graphical tool ([DrawFBP](#)), and in recent years the emphasis has been shifting to doing the diagramming on-line, and then generating the network code (we can see this progression with other flow tools on the market). Now that compile and run functions (for Java and C#) have been added to DrawFBP, the developer can do much of the development graphically, and interactively.

We are in the process of "growing" a [tutorial](#) which will introduce the reader to the basic concepts of FBP by means of a series of "steps", which use the diagramming tool DrawFBP to progressively expand an application diagram and generate and run code at many of those steps.

First description of Flow-Based Programming concepts (1971)

This is the IBM TDB (Technical Disclosure Bulletin) mentioned in the bibliography below. It can be found in a [database](#) which includes a number of "defensive" patents, and I have copied it onto this website for ease of reference. If the language is rather impenetrable, it is because it is written in "legalese" - insisted on by the legal eagles of IBM Canada in that long-ago era: [IBM TDB \(1971\)](#).

Here is a quote from my book (page 10):

Although the concepts are not well known [not really true any more!], they have actually been in the public domain for many years. The way this happened is as follows: in late 1970 or early '71 I approached IBM Canada's Intellectual Property department to see if we could take out a patent on the basic idea. Their

recommendation, which I feel was prescient, was that this concept seemed to them more like a law of nature, which is not patentable. They did recommend, however, that I write up a Technical Disclosure Bulletin (TDB), which was duly published and distributed to patent offices world-wide (Morrison 1971). A TDB is a sort of inverse patent – while a patent protects the owner but requires him or her to try to predict all possible variations on a concept, a TDB puts a concept into the public domain, and thereby protects the registering body from being restricted or impeded in the future in any use they may wish to make of the concept. In the case of a TDB, it places the onus on someone else who might be trying to patent something based on your concept to prove that their variation was not obvious to someone “skilled in the art”.

FBP Fora

- There is a [Google group](#) on FBP, with almost 800 members, and
- a Slack channel was set up, but the discussions have since been moved to Discord. An invitation to join the FBP discussion on Discord may be found on [Discussion on Discord](#).

FBP Tenets (open-ended)

- If work is boring, let a robot do it!
- Corollary: using a design tool should be fun!
- No one language should try to do *everything*!
- The world is asynchronous - don't try to force the systems we build into a synchronous framework!
- Information Packets are *things*
- The von Neumann paradigm is only appropriate within a *single* process! Duh!
- Any language or system that "talks data" should be able to talk to any other one that "talks data".
- A process needs to be able to have more than one input port, and needs to be able to choose which one to receive from!
- If a process's upstream connection fills up, the process feeding it will be suspended (unless the connection is marked "DropOldest") ("back pressure")

Links

1. [History](#)
2. [Concepts](#)
3. [Tutorial](#) (under development)
4. [Software](#):
 - [JavaFBP](#)
 - [JavaFBP Web Sockets](#)
 - [C#FBP](#)
 - [GoFBP](#)
 - [CppFBP \(C++ and Lua\)](#)
 - [JSFBP \(Archived\)](#)
 - [DrawFBP](#)
5. [Examples](#)
6. [FBP vs. FBP-inspired Systems](#)
7. [Comparison with OO](#)
8. [IBM Technical Disclosure Bulletin - "DSLm" - 1971](#)
9. [Comparison with other paradigms and methodologies](#)
10. [FBP and multithreading](#)
11. ["How An Arcane Coding Method From 1970s Banking Software Could Save The Sanity Of Web Developers Everywhere"](#), Gabe Stein, Aug., 2013, Co. Labs
12. [Tom Young's FBP web site, including categorization of FBP and FBP-like systems](#)
13. [PowerPoint presentation on FBP \(2007\)](#)
14. [Justin Bozonier's "knol" re FBP \(2009\)](#)
15. [C2-style FBP Wiki](#)
16. [Patterns in Flow-Based Programming](#)
17. [Bibliography](#)
18. [Chapters from the first edition - online](#)
19. [External links](#)



The origins of Flow Based Programming with J F

Paul Morrison



Watch on

Video Interview with J. Paul Rodker Morrison, Summer 2013

In addition, there are six Youtube videos about DrawFBP, illustrating a number of basic ("classical") FBP concepts (what we are now calling "FBP-inspired" or "FBP-like" systems do not necessarily contain all of these, although DrawFBP should be able to support most of these systems):

1. [DrawFBP video #1](#)
2. [DrawFBP video #2](#)
3. [DrawFBP video #3](#)
4. [DrawFBP video #4](#)
5. [DrawFBP video #5](#) - concept of "subnets", both at design and implementation time
6. [DrawFBP video #6](#) - simple interactive systems using WebSockets, with demo of JavaFBP-WebSockets (JavaFBP and HTML5)

Finally, a [video](#) presentation at a Meetup in Toronto in Sept. 2013 organized by Paul Tarvydas, CEO of Tarvydas-Sanford Controls Inc., and Dann Toliver, also containing a few jokes and reminiscences...

Google group on [Flow-Based Programming](#)

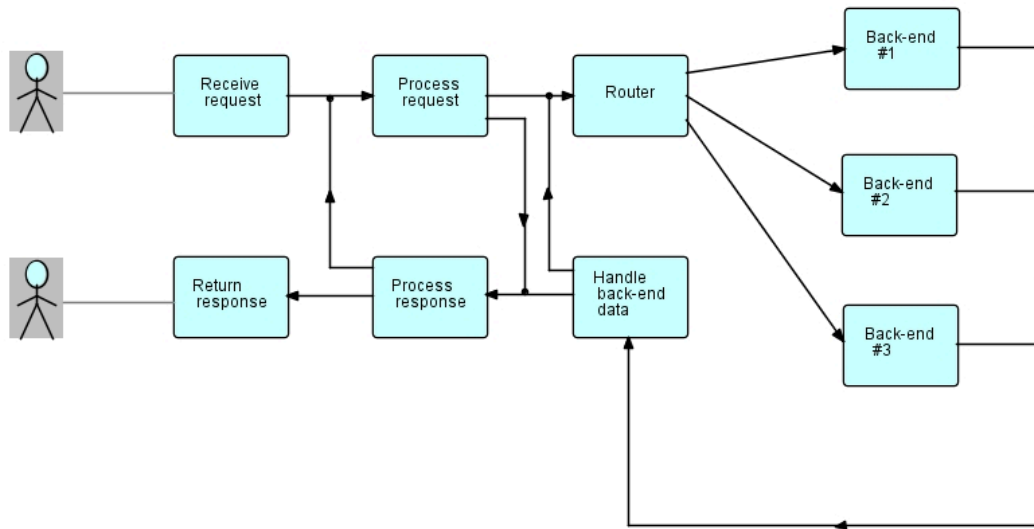
C2-style [Flow-Based Programming Wiki](#) (last maintained in 2010)

More general [FBP Wiki](#) by Vladimir Sibirov, et al., intended to cover both "classical" FBP and FBP-inspired approaches (still under development)

[FBP Specification wiki](#) - under development by Vladimir Sibirov

Twitter hashtag: [#FlowBased](#)

Discord invitation: [Invitation to Discord FBP channel](#)



Schematic Brokerage application (expand blocks, add caches, multiplexing, etc.)

J Paul Rodker Morrison

Content Copyright © J Paul Morrison. All rights reserved.